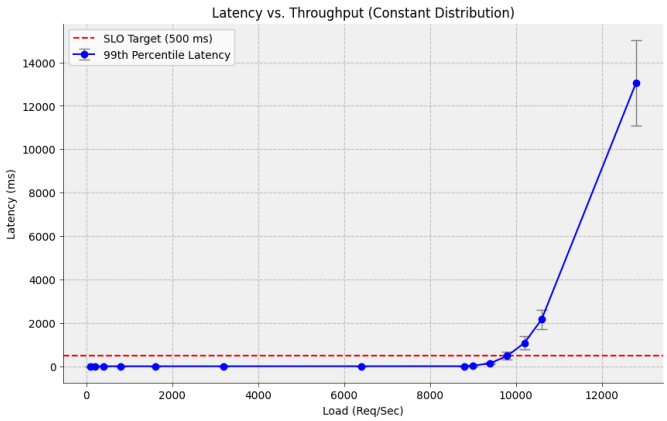
Lab 2: Simple Performance Analysis

By Chris Kim, Jared Jose, Ziyue Liu

A1. Analyzing Latency vs. Throughput

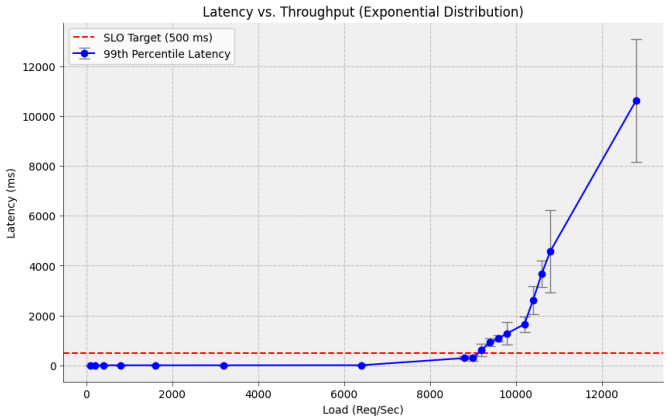


Latency vs. Load (Constant Arrivals)

• **Constant Distribution** Initial latency remains stable at low throughput under 10ms. But as throughput nears 9000 reqs/s, latency spikes dramatically. Below are key points.

- At 9000 req/s: Avg latency jumps to 28.92 ms (Std: 28.11)
- At 9400 req/s: 139.22 ms (Std: 33.09), a sudden spike occurs.
- At 10800 req/s: 2,160 ms (Std: 448.37), over 10% requests missed.
- At 12800 req/s: 13,060 ms (Std: 1971.04), complete overload.

The pattern suggests that constant arrivals allow predictable performance until around 9000 req/s (the arrival rate) starts to exceed service rate (μ). Here we can assume μ is roughly 9000 req/s. Moreover, we can assume 9400 req/s to be the knee point where the spike occurs. At 10800 req/s, system is no longer capable of finishing all the requests, which we determine as max point.



Latency vs. Load (Exponential Arrivals)

• **Exponential Distribution** introduces burstiness, meaning queues tend to fill quicker. Below are the key points.

- At 8800 req/s: Latency jumps to 290.20 ms (Std: 62.96), much higher than equivalent load under constant distribution.
- At 9400 req/s: 933.57 ms (Std: 152.88), a sudden spike occurs..
- At 10400 req/s: 2612 ms (Std: 572.691), over 10% requests missed.
- At 12800 req/s: 10,632 ms (Std: 2468.50), overload more noise.

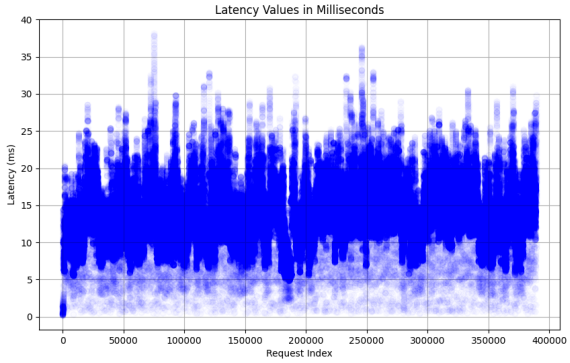
Due to bursty arrivals, queue build up begins around 8800 req/s, suggesting μ ≈ 8800 req/s under this distribution, also resulting in high variability due to exponential distribution. Similar to constant distribution, knee point occurs around 9400 req/s. Max point occurs a little earlier as well, around 10400 req/s, showing the bursty nature shifted all points.

Analysis One of the key differences between the distribution was its variability. Under a constant load, variance remains low until saturation (~9000 req/s), after which tail latencies spike (800+ ms to seconds) due to queue buildup. But with exponential arrivals, variance increases earlier due to fluctuating queue lengths, leading to intermittent congestion and greater deviations (152.88 ms at 9400 req/s vs. 33.09 ms under constant arrivals). Instead of a smooth transition, the system oscillates between idle and congested states, amplifying variability (also shown in plotting). In both cases, latency surges past the knee point as requests exceed the server's processing rate, causing queues to grow uncontrollably. Each new task waits longer, worsening delays. However, with constant arrivals, queues remain persistently overloaded, leading to sustained degradation. Exponential arrivals, while bursty, allow brief recovery periods, slightly easing queue pressure. Yet, they ultimately cause faster and more volatile latency spikes than constant arrivals.

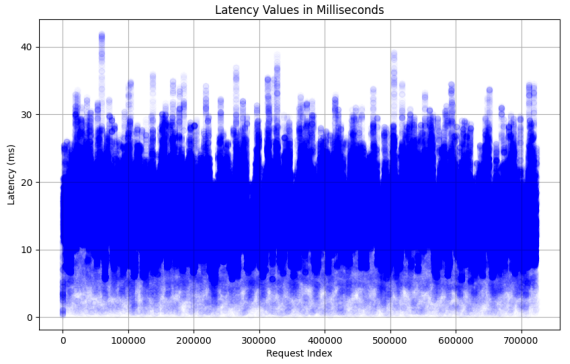
Discussion Before reaching the knee point, the arrival pattern has little impact on latency. However, beyond this threshold, exponential arrivals degrade performance faster—users experience increased delays sooner than with constant arrivals. Assuming users expect a response within 500ms, this threshold occurs earlier for exponential patterns (~9300 req/s) than for constant ones (~9800 req/s). For real-world applications with diurnal traffic patterns (e.g., restaurant lookups peaking at lunch/dinner for our restaurant review app), we need to anticipate loads and scale accordingly as queues can back up and lead to multi-second delays; harming user experience and retention. Therefore, scaling must be proactive, not reactive especially due to the nature of our application.

A2. Bottleneck Analysis

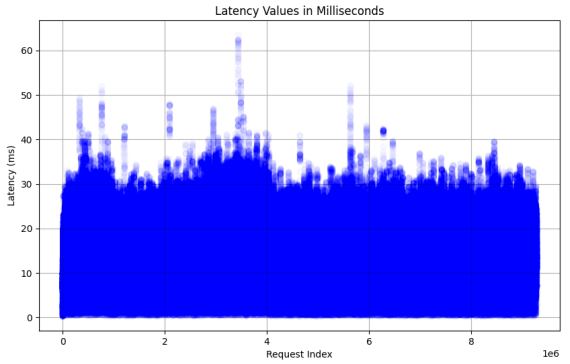
Setup We have two latency metrics: Kubernetes logs for individual microservice latency and wrk2 output for total request latency. Using these, we can identify the bottleneck service—Frontend or Detail. To investigate, we ran wrk2 at the maximum rate from Assignment 1, beyond the knee point, where runaway latencies should occur. As latency rises due to queue overload, the bottleneck service will show a continual increase in latency, which we can verify using gRPC logs for the Detail microservice.



1 minute workload @ 12800 req/s

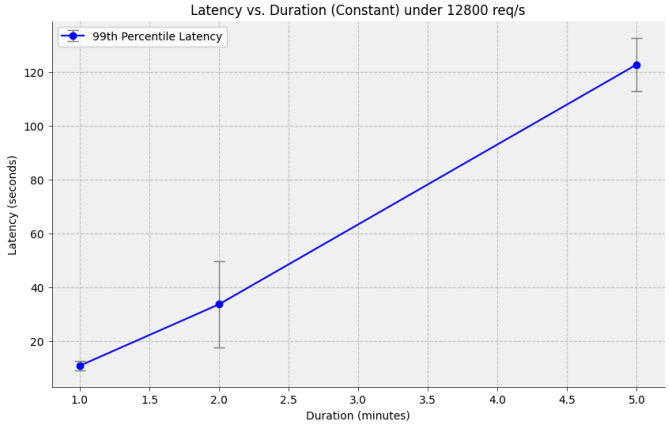


2 minute workload @ 12800 req/s



5 minute workload @ 12800 req/s

Conclusions As we increased throughput and workload duration, gRPC latency (retrieved from Kubernetes logs) remained relatively stable, with the majority of requests falling within the 20ms to 30ms range, though with significant fluctuations. This pattern held across all durations—1m, 2m, and 5m workloads. However, when observing end-to-end response time from wrk2 outputs, we saw a linear increase in latency as workload duration increased, indicating that the frontend was likely the bottleneck. The frontend struggled to process and return responses, leading to overall latency.By fixing the request rate, we were able to push the system beyond its knee point, making the increase in latency more dramatic. This resulted in significant increases in 99% latency in wrk2, but this increase was not reflected in the gRPC logs. The gRPC logs for the Detail service showed variability, but no continuous rise in latency, suggesting that the Detail service was not the bottleneck. Given that the overall latency increased while the Detail service remained stable, the increasing latency must be due to the frontend. The service queue of the bottlenecked frontend service must have been continually growing as the workload duration increased, further supporting that the frontend is the primary source of the latency spike.



End-to-End Response Time over Time

Furthermore, we observed instances where `postDetail` requests temporarily reduced `getDetail` latency. Since `getDetail` requires the frontend to process restaurant detail in JSON (while `postDetail`) does it for backend, it further seemed to support the fact that frontend is the overhead.

02/04/2025	3:45:35	detail.DetailService/GetDetail	5279
02/04/2025	3:45:35	detail.DetailService/PostDetail	5475
02/04/2025	3:45:35	detail.DetailService/GetDetail	4370
02/04/2025	3:45:35	detail.DetailService/GetDetail	4394
02/04/2025	3:45:35	detail.DetailService/GetDetail	1161
02/04/2025	3:45:35	detail.DetailService/GetDetail	3697

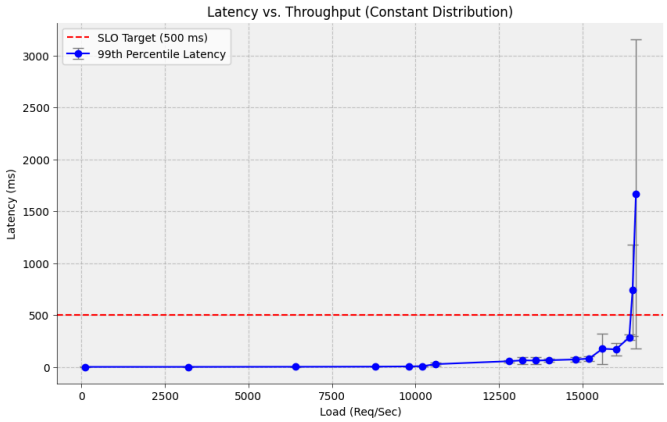
Temporary Reduction

Both codes are nearly identical in structure, so most of the overhead likely originates from gRPC routing and JSON encoding/structuring. It could be the case that the backed queue for frontend was getting a break while `postDetail` was being processed by the backend.

A3. Explore the Impact of Service Scaling

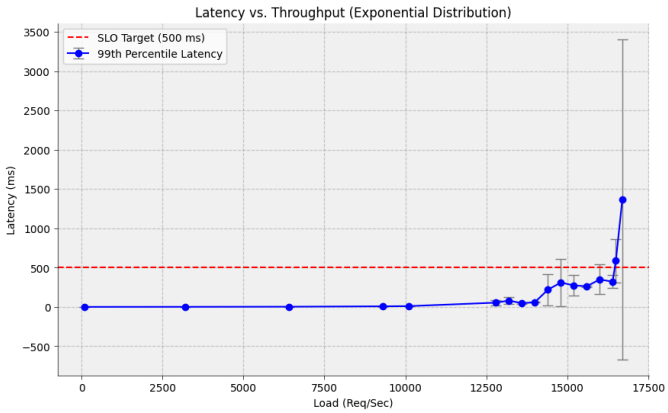
1. Impact of Two Instances

• **Constant Distribution** The added instance immediately improved latency at higher request rates, staying below 500 ms up to ~16000 req/s before degrading. However, the onset of performance degradation is more gradual, showing better fault tolerance and load distribution. Below shows the plot.



Constant Arrivals with Two

• **Exponential Distribution** Similarly, the system sustained lower latencies. The performance improved, sustaining lower latencies up to ~15600 req/s. However, variance increased by a larger margin with unpredictable latency spikes, indicating system strain from queueing delays or load imbalance.

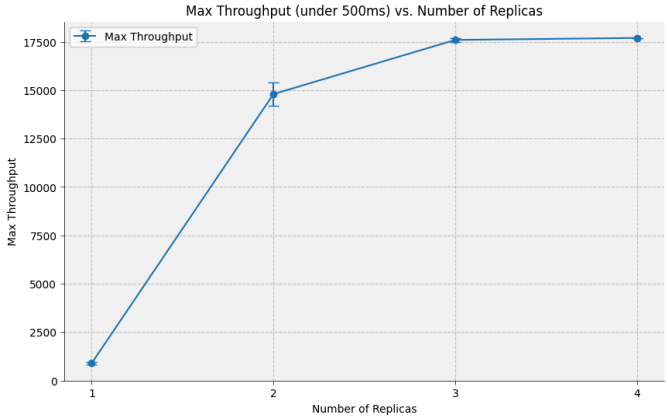


Exponential Arrivals with Two

Both distributions delayed the onset of the knee point, however exponential distribution led to slightly earlier onset as well as higher variability due to bursty nature. Constant distribution on the other hand had clear saturation points although it also had significant variability.

2. Diminishing Returns of Increased Instances

Setup We incrementally increased the number of replicas to observe throughput maintaining tail latency below 500ms. For brevity, we assumed exponential distribution. The workload duration was extended to 2 minutes to ensure statistically significant results, with over 90% of requests completing successfully. We then identified the maximum request rate that met the latency threshold.



Max Throughput by Number of Replicas

3. Reason for Diminished Returns

The effects quickly diminished after 2 replicas, and by the 4th replica, there was virtually no difference. This is possibly due to bottlenecks shifting elsewhere. We tried adding more replicas for detail, but it didn't show improvements. Other reasons could be network latency, or overhead from load balancing overhead from added instances.

4. Determining Shifted Bottleneck

To determine shifting bottlenecks, we might have to track system resources. It will be quite difficult to go through all of them, so it might be important to have some idea of the likely culprits. Firstly, I would try to increase the detail service to rule it out (which we did), checking disk I/O, locking mechanisms in the gRPC code could be a likely culprits.

Lastly, since the load generator could also be the bottleneck, running multiple load generators also might be a good idea to determine if it's a bottleneck. If it increases, it means the load generator isn't generating the load at the listed rates.

B1: Optimizations

Proposed Changes

• **Opt1: Lockless Maps** One of the suspected major contentions on our implementations were retrieving the restaurant details using maps and invoking locks on every access. By using the `sync.Map` package which allows lockless access [1], we can see overhead reduction from locks. The store operations are amortized at a constant time, and given our app is mostly retrieval (we assume 80% `getDetail`, 20% `postDetail`), we can expect improvement to be significant.

• **Opt2: Reuse Connection** gRPC also was thought to be a major point of overhead upon code inspection. The official gRPC guide[3] suggests using keep alive pings to reuse the connections (rather than reconnecting every request) and using streaming RPCs to avoid RPC initiation.

• **Opt3: json Library** As we've observed in A2, we hypothesized that JSON structuring might be one of the major sources of overhead as well. The `jsoniter` library provides faster parsing (up to 6 times more according to their website [2]) using single pass scan and minimum allocation.

• **gRPC proxy load balancing Optimization** Kubernetes is designed to load balance single, stateless requests, which doesn't efficiently handle long-lived connections like those used by gRPC (which operates over HTTP/2). This results in suboptimal performance, particularly for streams in TCP connections. To mitigate this, we propose introducing a proxy load balancer between the load generator and the frontend (the bottleneck discussed in Section II). We selected Istio, a lightweight load balancer similar to Nginx, to address the issue.

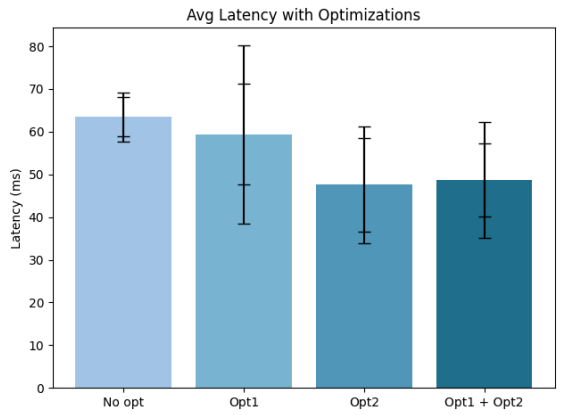
1. Effect Under Different Conditions

• **Low Loads**: In loads under the service rate (around 9000 req/s), there seemed to be some improvement particularly for gRPC connection reuse. However, the difference was very little and due to variability, it was difficult to determine whether optimization was actually working.

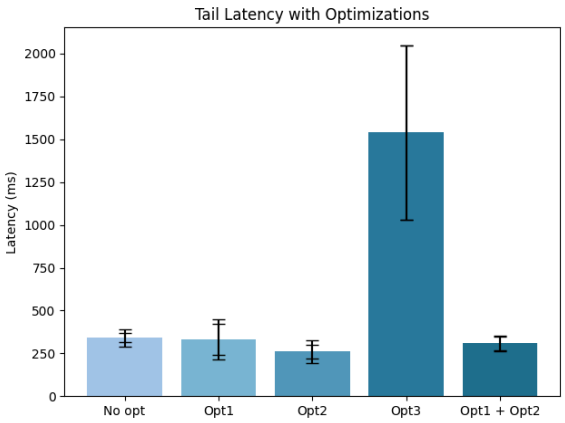
• **Knee Point**: With a fixed load of 9400, right under max point, there was noticeable improvement for connection reuse, both on average and tail latencies. Opt1 has shown negligible improvement, possibly due to the backend not being the bottleneck. Opt3 seemed to worsen the latency.

2. Improvement On Previously Struggled

The average latency had seen noticeable improvement in the previously struggled knee point, but the tail latency didn't show significant improvement, possibly implying tail latencies are inevitable result of limitations of the system itself.



Average Latency by Optimizations



Tail Latency by Optimizations

3. Trade-offs or Limitations

• Overall, the main limitation was that it didn't have much effect on tail latencies, but rather reduced average latency.

• **Lockless Maps**: Due to amortized nature, sync.Map has limitation when majority of operations are writes.

• **Reuse Connection**:Connection reuse strategy can lead to memory leaks if connections aren't properly managed and closed when no longer needed - idle connections still consume system resources and file descriptors.

• **Third Party Load Balancing**: Proxy load balancing introduces latency, and with only three frontend servers, the overhead outweighs its benefits.

---

[1] https://pkg.go.dev/sync
[2] https://jsoniter.com/benchmark.html#optimization-used
[3] https://grpc.io/docs/guides/performance/